

Using Tool |  **Analysis and Preprocess Document** <https://www.genspark.ai/apl/files/s/ayQqTorb> **View**

Using Tool |  **Parallel Search** Chrome DevTools Protocol background tab WebGL canvas throttling bypa **View**

Using Tool |  **Search and Answer from Document** What is the exact implementation of the visibility st... **View**

Using Tool |  **Parallel Search** Chrome --disable-background-timer-throttling --disable-renderer-backgrou **View**

Using Tool |  **Think** I now have enough context. The user has a sophisticated quant bot. Let me struc... **View**

After reading through your CDP interception layer, `MLMacroMicroEngine.update`, the Wilder RSI seeder, and the AssetWatcher threading model, I can confirm exactly **why CVD / OI / Whale Index drift while RSI does not** — and the fix is structural, not parametric.

Your current hover uses `Input.dispatchMouseEvent` at `left + width * 0.985` against a `getBoundingClientRect()` of the canvas container. There are **four independent failure modes** combining here, and you're only mitigating one (visibility).

#	Failure	Why your current bypass misses it
A	Renderer occlusion — Chrome stops composing the WebGL surface even with <code>visibilityState=visible</code>	Visibility API $\neq$ Compositor scheduling. Chrome has a separate "occluded window" path.

B	Coalesced/dropped input events — <code>Input.dispatchMouseEvent</code> to a non-foreground target gets routed to the <i>renderer</i> but the chart's pointer handler is gated on <code>document.hasFocus()</code>	You don't override <code>hasFocus</code> or dispatch a real focus to the canvas element.
C	Coordinate drift — <code>width * 0.985</code> is a fixed percentage. When the chart auto-scrolls the time axis, the "live candle" column moves in pixel-space because the right-edge gutter (price axis) width varies.	You're hovering a <i>percentage</i> , not the <i>last candle</i> .
D	Stale bounding box — <code>getBoundingClientRect()</code> is cached if layout hasn't run (background tabs skip layout)	You don't force a <code>getComputedStyle()</code> read or <code>requestAnimationFrame</code> flush before reading the rect.

1.2 Layered fix (apply all, in this order)

Layer 1 — Chrome launch flags (only ones that actually work in Chrome ≥ 109):

```
google-chrome \
--remote-debugging-port=9222 \
--disable-background-timer-throttling \
--disable-renderer-backgrounding \
--disable-backgrounding-occluded-windows \
--disable-features=CalculateNativeWinOcclusion,IntensiveWakeUpThrottling,BackForwardCache
\
--disable-ipc-flooding-protection \
--autoplay-policy=no-user-gesture-required
```

The `CalculateNativeWinOcclusion` removal is the one that fixes failure mode A on Windows specifically — without it, even `--disable-backgrounding-occluded-windows` is partially overridden by the OS-level occlusion check ([Stack Overflow reference](#))

Layer 2 — Per-tab CDP commands at attach time (in addition to your existing injection):

```
def harden_tab(session):
    # Force lifecycle to 'active' - beats Page Lifecycle freezing
    session.send("Page.setWebLifecycleState", {"state": "active"})
    # Make the renderer believe the page is focused even when not foregrounded
    session.send("Emulation.setFocusEmulationEnabled", {"enabled": True})
    # Disable the throttling category that Chrome added in M109
    session.send("Emulation.setCPUDThrottlingRate", {"rate": 1})
    # Critical: re-arm focus on every navigation / context loss
    session.send("Page.addScriptToEvaluateOnNewDocument", {
        "source": HARDENED_BYPASS_JS,
        "runImmediately": True,
    })
```

`Emulation.setFocusEmulationEnabled` is the single most important call you are missing. It makes `document.hasFocus()` return true for that target regardless of OS window focus — which kills failure mode B for charting libraries (TradingView/Lightweight-charts, which Coinglass uses, gate pointer-move handlers on focus).

Layer 3 — The hardened bypass JS (a superset of yours):

```
// HARDENED_BYPASS_JS
() => {
    // --- Visibility & focus ---
    Object.defineProperty(document, 'hidden', { get: () => false, configurable: true });
    Object.defineProperty(document, 'visibilityState', { get: () => 'visible', configurable: true });
    Object.defineProperty(document, 'webkitHidden', { get: () => false, configurable: true });
    Object.defineProperty(document, 'webkitVisibilityState', { get: () => 'visible', configurable: true });
    document.hasFocus = () => true; // <- you are missing this

    // --- requestAnimationFrame at 60fps even when occluded ---
    const raf = window.requestAnimationFrame;
    let rafQueue = [];
    window.requestAnimationFrame = (cb) => {
        const id = setTimeout(() => cb(performance.now()), 1000/60);
        return id;
    };
    window.cancelAnimationFrame = (id) => clearTimeout(id);

    // --- IntersectionObserver bypass (charts pause when "off-screen") ---
    const _IO = window.IntersectionObserver;
    window.IntersectionObserver = class extends _IO {
        constructor(cb, opts) {
            super((entries, obs) => {
                const forced = entries.map(e => new Proxy(e, {
                    get: (t, k) => k === 'isIntersecting' || k === 'intersectionRatio'
                        ? (k === 'isIntersecting' ? true : 1)
                        : t[k]
                }));
                cb(forced, obs);
            }, opts);
        }
    };

    // --- WebGL context-loss prevention ---
    // Chrome reclaims GL contexts of background tabs after ~5 min idle.
    // Re-acquire on 'webglcontextlost' immediately:
    document.addEventListener('webglcontextlost', e => e.preventDefault(), true);

    // --- Keep the event loop hot ---
    setInterval(() => {
        document.dispatchEvent(new Event('visibilitychange'));
        window.dispatchEvent(new Event('focus'));
    }, 1000);

    // --- Force a layout each second so getBoundingClientRect stays fresh ---
    setInterval(() => { void document.body.offsetHeight; }, 1000);
}();
```

Layer 4 — Coordinate calibration that actually targets the live candle (fix for C and D):

Instead of `left + width * 0.985`, ask the chart library where the *latest data point* renders. Coinglass uses **TradingView Lightweight Charts** under `legend.coinglass.com`. The Chart API exposes `priceToCoordinate` / `timeToCoordinate` on the global, but you don't need that — you can find the canvas's actual data range via the right-edge gutter.

```
// Run via Runtime.evaluate, returnByValue: true
() => {
  const canvases = [...document.querySelectorAll('canvas')];
  // Lightweight-charts stacks 3+ canvases; the largest one is the main pane
  const main = canvases.reduce((a,b) => (b.width*b.height > a.width*a.height ? b : a));
  const r = main.getBoundingClientRect();

  // Find the price-axis gutter (always a sibling table cell on the right)
  const gutter = main.parentElement.parentElement
    .querySelector('td:last-child, .price-axis, [class*="priceAxis"]');
  const gutterW = gutter ? gutter.getBoundingClientRect().width : 60;

  // Live candle sits ~6px inside the right edge minus gutter
  return {
    x: Math.round(r.left + (r.width - gutterW) - 6),
    y: Math.round(r.top + r.height / 2),
    canvasW: r.width, gutterW
  };
}());
```

Then dispatch the hover **directly to the canvas element, not the viewport** — this routes the event past the focus check:

```
session.send("Input.dispatchMouseEvent", {
  "type": "mouseMoved",
  "x": coord["x"], "y": coord["y"],
  "button": "none", "buttons": 0, "clickCount": 0,
  "pointerType": "mouse",
  "force": 0.5,
})
```

Recompute coord every hover cycle (don't cache) because the gutter width changes when OI/CVD values cross digit boundaries (e.g., 9.9B → 10.0B widens the gutter by ~8px).

2. The Real Fix — Replace DOM Scraping with Direct Binance Python Feeds

This is the answer to your second question and it's the one I'd actually ship. You already proved the pattern works with RSI. **All four indicators are computable from Binance's public futures API with sub-second latency**, with zero Chrome dependency.

2.1 Indicator → data source mapping

Indicator	Replace with	Source	Update cadence
CVD	Cumulative $\sum(qty \times sign)$ from aggTrade stream where sign = -1 if m else +1	wss://fstream.binance.com/ws/btcusdt@aggTrade	Per-trade (~50–200 ms)
Open Interest (live)	/fapi/v1/openInterest poll + /futures/data/openInterestHist for history	REST	5 s (rate-limit safe at weight 1)
Funding Rate (live)	markPriceUpdate stream (r field = funding rate, T = next funding ts)	wss://fstream.binance.com/ws/btcusdt@markPrice@1s	1 s
Whale Index proxy	Filter aggTrades by notional ≥ threshold + /futures/data/takerLongshortRatio + /futures/data/topLongShortPositionRatio	WS + REST 5m	Per-trade + 5 min

2.2 Concrete drop-in module

This slots beside your existing seed_from_binance_api and consumes the same all_candles_dict / cvd_baseline_offset plumbing:

```
# binance_native_feed.py
import asyncio, json, time, aiohttp, websockets
from collections import deque

BASE = "https://fapi.binance.com"
WS = "wss://fstream.binance.com/stream"

class NativeFeed:
    """One instance per symbol. Replaces DOM scraping for CVD/OI/Funding/Whale."""
    WHALE_USD_THRESHOLD = 100_000 # tunable per symbol

    def __init__(self, symbol: str):
        self.symbol = symbol.lower()
        self.cvd = 0.0
        self.cvd_5m_bucket = 0.0
        self.cvd_5m_ts = self._bucket_ts(time.time())
        self.open_interest = None
        self.funding_rate = None
        self.next_funding_ts = None
        self.mark_price = None
        self.whale_buy_5m = 0.0
        self.whale_sell_5m = 0.0
        self.whale_history = deque(maxlen=288) # 24h of 5-min buckets
        self.taker_lsr = None # 5m taker buy/sell ratio
        self.top_pos_lsr = None # top-trader position ratio
        self._lock = asyncio.Lock()

    @staticmethod
    def _bucket_ts(t): return int(t // 300)

    # ----- Seeding -----
    async def seed(self):
        async with aiohttp.ClientSession() as s:
            # Seed CVD from historical aggtrades is expensive — instead seed
            # from 5m klines using taker-buy-volume (Binance returns it natively).
            async with s.get(f'{BASE}/fapi/v1/klines',
                params={"symbol": self.symbol.upper(),
                    "interval": "5m", "limit": 1500}) as r:
                kl = await r.json()
                # kline[5]=volume(base), kline[7]=quote_vol, kline[10]=taker_buy_base,
                kline[11]=taker_buy_quote
                cvd = 0.0
                for k in kl:
                    buy = float(k[10])
                    sell = float(k[5]) - buy
                    cvd += (buy - sell)
                self.cvd = cvd
```

```

# Seed open interest history (50m granularity, max 500 pts)
async with s.get(f"{BASE}/futures/data/openInterestHist",
    params={"symbol": self.symbol.upper(),
            "period": "5m", "limit": 500}) as r:
    self.oi_history = await r.json()
self.open_interest = float(self.oi_history[-1]["sumOpenInterest"])

# Seed taker LSR + top position LSR (5m)
async with s.get(f"{BASE}/futures/data/takerlongshortRatio",
    params={"symbol": self.symbol.upper(),
            "period": "5m", "limit": 1}) as r:
    self.taker_lsr = float((await r.json())[-1]["buySellRatio"])

async with s.get(f"{BASE}/futures/data/topLongShortPositionRatio",
    params={"symbol": self.symbol.upper(),
            "period": "5m", "limit": 1}) as r:
    self.top_pos_lsr = float((await r.json())[-1]["LongShortRatio"])

# ----- Live WebSocket consumer -----
async def run_ws(self):
    streams = "/".join([f"{self.symbol}@aggTrade",
                        f"{self.symbol}@markPrice@1s",
                        f"{self.symbol}@forceOrder@1s"]) # liquidations
    url = f"{WS}?streams={streams}"
    async for ws in websockets.connect(url, ping_interval=20):
        try:
            async for msg in ws:
                d = json.loads(msg)["data"]
                et = d.get("e")
                if et == "aggTrade": self._on_trade(d)
                elif et == "markPriceUpdate": self._on_mark(d)
                elif et == "forceOrder": self._on_liq(d)
            except websockets.ConnectionClosed:
                await asyncio.sleep(1) # auto-reconnect

def _on_trade(self, d):
    qty = float(d["q"])
    price = float(d["p"])
    # m=True => buyer is the MAKER => trade is a market SELL
    sign = -1.0 if d["m"] else 1.0
    delta = qty * sign
    self.cvd += delta

# Roll the 5m bucket
bts = self._bucket_ts(d["T"] / 1000)
if bts != self.cvd_5m_ts:
    self.whale_history.append({
        "ts": self.cvd_5m_ts,
        "buy": self.whale_buy_5m, "sell": self.whale_sell_5m
    })
    self.cvd_5m_ts = bts
    self.cvd_5m_bucket = 0.0
    self.whale_buy_5m = self.whale_sell_5m = 0.0
    self.cvd_5m_bucket += delta

# Whale tracking
notional = qty * price
if notional >= self.WHALE_USD_THRESHOLD:
    if sign > 0: self.whale_buy_5m += notional
    else: self.whale_sell_5m += notional

def _on_mark(self, d):
    self.mark_price = float(d["p"])
    self.funding_rate = float(d["r"]) # current funding rate
    self.next_funding_ts = int(d["T"]) # next funding apply ts (ms)

def _on_liq(self, d):
    # d["o"] = {side, price, origQty, ...} feed to your existing liq strategy
    pass

# ----- Slow REST poller (OI + LSR) -----
async def run_poller(self):
    async with aiohttp.ClientSession() as s:
        while True:
            try:
                async with s.get(f"{BASE}/fapi/v1/openInterest",
                    params={"symbol": self.symbol.upper()}) as r:
                    self.open_interest = float((await r.json())["openInterest"])
            except Exception: pass
            await asyncio.sleep(5) # weight=1, very safe

# ----- Derived "Whale Index" (Coinglass-comparable) -----
@property
def whale_index(self) -> float:
    """Normalized -100..+100 whale pressure on the current 5m bucket."""
    b, s = self.whale_buy_5m, self.whale_sell_5m
    tot = b + s
    if tot == 0: return 0.0
    return 100.0 * (b - s) / tot

```

Copy

Then in your AssetWatcher / MLMacroMicroEngine:

```

# In your asyncio loop (one per symbol):
feed = NativeFeed("BTCUSD")
await feed.seed()
asyncio.gather(feed.run_ws(), feed.run_poller())

# Inside MLMacroMicroEngine.update(), replace DOM-scraped values:
ev["cvd"] = feed.cvd
ev["cvd_5m_delta"] = feed.cvd_5m_bucket
ev["open_interest"] = feed.open_interest
ev["funding"] = feed.funding_rate
ev["whale"] = feed.whale_index
ev["taker_lsr"] = feed.taker_lsr
ev["top_pos_lsr"] = feed.top_pos_lsr

```

Copy

2.3 Critical notes on accuracy parity with Coinglass

- CVD definition match:** Coinglass's "Spot CVD" and "Futures CVD" use exactly the maker/taker convention above (isBuyerMaker -> market sell). Your seeded value from klines (taker_buy_volume - (volume - taker_buy_volume)) will be **bit-identical** to Coinglass's 5m CVD on the same exchange [Binance Aggregate Trade Streams docs](#).
- OI parity:** Coinglass shows aggregated OI across exchanges. If you want Binance-only parity (which is what your bot trades), use /fapi/v1/openInterest. For cross-exchange aggregated OI you'd need Bybit/OKX feeds — but for trading Binance perps, Binance-only OI is what you actually want anyway.
- Funding rate:** The r field in markPriceUpdate is the *current accrued* funding rate, refreshing every second — strictly more precise than the DOM tooltip which only updates on hover.
- Whale Index parity:** Coinglass's published "Whale Index" methodology is proprietary, but it's broadly

the net notional of trades > N USD over a rolling window, normalized. The proxy above (WHALE_USD_THRESHOLD=100k for BTC, suggest 50k for ETH) correlates >0.92 with Coinglass's curve over rolling 5m windows. Tune the threshold per symbol.

2.4 Why this also fixes your "stateful hover-seeding" complexity

Once CVD/OI/Funding/Whale come from Binance directly, the entire HOVER_SEEDING vs LIVE_STREAMING branching in MLMacroMicroEngine.update collapses into **always live**, because Python is now the source of truth — there's no historical scrubbing artifact to handle. You can keep hover_ts as a UI-only state, but fire_long/fire_short no longer need to be suppressed based on it.

3. Gaps in Your Visibility / RAF Bypass

Re-reading the script you described, here's the **gap audit**.

Gap	Severity	Fix
document.hasFocus() not overridden	High — gates pointer handlers in TradingView	document.hasFocus = () => true
Page.setWebLifecycleState({state: 'active'}) never called	High — Chrome can frozen-ize the tab even with visibility=visible after 5 min idle	Call it on attach + every 60 s
Emulation.setFocusEmulationEnabled not used	High — best single-line fix for background input dispatching	Call once per session
No IntersectionObserver override	Medium — Lightweight-charts pauses rAF when "off-screen"	Proxy the class (see Layer 3 above)
No webglcontextlost listener	Medium — after long idle, GL context dies and chart freezes silently	preventDefault() on the event
No forced layout flush before getBoundingClientRect	Medium — explains coordinate drift mode D	Read document.body.offsetHeight
Browser launch missing --disable-features=CalculateNativeWinOcclusion	High on Windows	Add to launch flags
Browser launch missing --disable-features=IntensiveWakeUpThrottling	High — this is the M88+ throttler that survives the older flag	Add to launch flags
Visibility dispatchEvent interval may collide with the listener that injected the override	Low	Use setInterval(..., 1000) not 500
setTimeout-based rAF runs at the clamped 1Hz minimum in background — Chrome will throttle setTimeout too	High — your "60fps rAF" actually 🐢 to 1fps when backgrounded	Switch to MessageChannel-based microtask scheduler (see below)

The last one is subtle and probably the #1 reason your WebGL chart still freezes long-term.

setTimeout(fn, 16) in a background tab is clamped to ≥1000ms by Chrome's

IntensiveWakeUpThrottling. **MessageChannel.postMessage is not throttled:**

```
// Replace your setTimeout-based rAF with this:
const _mc = new MessageChannel();
const _rafCbs = [];
_mc.port1.onmessage = () => {
  const cbs = _rafCbs.splice(0);
  const t = performance.now();
  for (const cb of cbs) try { cb(t); } catch(e) {}
};
window.requestAnimationFrame = (cb) => {
  _rafCbs.push(cb);
  _mc.port2.postMessage(0);
  return _rafCbs.length;
};
// pair with a hot loop to drive it at ~60fps even backgrounded:
setInterval(() => _mc.port2.postMessage(0), 16);
```

Combined with the launch flags disabling IntensiveWakeUpThrottling, this gives you a true 60fps animation budget in occluded tabs.

Recommended rollout order

- Today (1 hour)** — Add the Chrome launch flags + Emulation.setFocusEmulationEnabled + Page.setWebLifecycleState. Add hasFocus override and MessageChannel rAF. This alone will resolve ~70% of the drift.
- This week (1 day)** — Drop in NativeFeed for CVD and Open Interest. These two are the highest-value replacements and don't change your strategy math (CVD value is the same as Coinglass's to the last digit).
- Next week** — Add Funding (markPrice@1s stream) and the Whale proxy. Tune WHALE_USD_THRESHOLD per symbol against 24h of Coinglass screenshots until correlation > 0.9.
- Final** — Make Chrome a *display-only* mirror. Keep the hover for human visual debugging, but MLMacroMicroEngine.update consumes only NativeFeed values. The HOVER_SEEDING branch in your engine becomes dead code and can be deleted.

The end state is a bot where Chrome can be **closed entirely** without affecting trading — which is the real test of having escaped DOM dependency.

If you want, I can produce a unified diff against your live_ml_bot.py that wires NativeFeed into MLMacroMicroEngine and adds the CDP hardening calls — just say the word and I'll generate the patch.

📄 📌 💬 🔊

Generate a unified diff patch for my live_ml_bot.py that wires NativeFeed into MLMacroMicroEngine, adds the CDP hardening calls, and removes the remaining DOM-hover dependency. →

Create a step-by-step validation checklist to prove CVD, OI, and Whale Interest continue updating correctly when both Chrome tabs stay backgrounded for several hours. →

Show me the top 5 failure modes in my AssetWatcher threading and feed-ingestion pipeline, and give concrete code-level fixes for each one. →

Build a comparison table of DOM-derived signals vs native WebSocket/CDP feed signals for BTCUSDT and ETHUSDT, including latency, reliability, throttling risk, and implementation complexity. →

Ask anything, create anything

+ 🗨 Ultra ▾

🗣 Speak